Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date: 2/4/2026**	**Enrolment No: 92301733059**

Aim: To recommend an item to the user using content-based filtering approach

IDE: Google Colab

Theory:

There are a lot of applications where websites collect data from their users and use that data to predict the likes and dislikes of their users. This allows them to recommend the content that they like. Recommender systems are a way of suggesting or similar items and ideas to a user's specific way of thinking.

Recommender System is different types:

- **Collaborative Filtering:** Collaborative Filtering recommends items based on similarity measures between users and/or items. The basic assumption behind the algorithm is that users with similar interests have common preferences.
- **Content-Based Recommendation:** It is supervised machine learning used to induce a classifier to discriminate between interesting and uninteresting items for the user.

Content-Based Recommendation System: Content-Based systems recommends items to the customer similar to previously high-rated items by the customer. It uses the features and properties of the item. From these properties, it can calculate the similarity between the items.
In a content-based recommendation system, first, we need to create a profile for each item, which represents the properties of those items. From the user profiles are inferred for a particular user. We use these user profiles to recommend the items to the users from the catalog.

Item profile:
In a content-based recommendation system, we need to build a profile for each item, which contains the important properties of each item. For Example, If the movie is an item, then its actors, director, release year, and genre are its important properties, and for the document, the important property is the type of content and set of important words in it.
Let's have a look at how to create an item profile. First, we need to perform the TF-IDF vectorizer, here TF (term frequency) of a word is the number of times it appears in a document and The IDF (inverse document frequency) of a word is the measure of how significant that term is in the whole corpus.

User profile:
The user profile is a vector that describes the user preference. During the creation of the user's profile, we use a utility matrix that describes the relationship between user and item. From this information, the best estimate we can decide which item the user likes, is some aggregation of the profiles of those items.
Advantages and Disadvantages:

- **Advantages:**
  - No need for data on other users when applying to similar users.
  - Able to recommend to users with unique tastes.
  - Able to recommend new & popular items

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date: 2/4/2026**	**Enrolment No: 92301733059**

- Explanations for recommended items.

- **Disadvantages**:
  - Finding the appropriate feature is hard.
  - Doesn't recommend items outside the user profile.

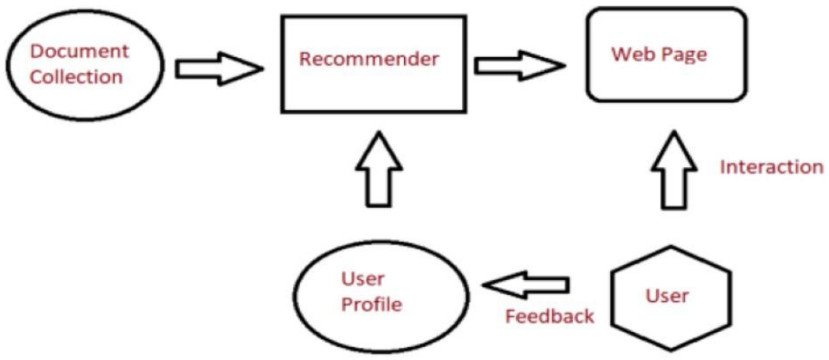


Fig: Recommender System

Pre Lab Exercise:

1. **When can you use content-based recommendation system approach?**
   We can use a content-based approach when:
   - You know the "Ingredients": You have clear information about the items (like movie genres, actors, or book categories) to help make matches.
   - The Item is New: You want to recommend a brand-new item that no one has rated yet, but you know its features.
   - The User is "Unique": The user has very specific tastes and you don't want their recommendations to be influenced by what other people like.
   - You want to Explain "Why": You want to be able to tell the user: "We suggested this because you liked a similar Action movie yesterday."
   - You want to avoid "The Crowd": You want to suggest things based on personal fit rather than just what is currently popular or trending.
2. **Write advantages of content-based filtering approach**
   - User Independence: The system builds a profile for each user independently; it does not require data from other users to generate recommendations.
   - Transparency and Explain ability: Recommendations are easily justified by showing the specific features (like genre or actor) that triggered the suggestion.
   - No "Cold Start" for Items: New items can be recommended immediately as soon as their attributes are defined, even before anyone has rated them.

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date: 2/4/2026**	**Enrolment No: 92301733059**

- Captures Niche Interests: It excels at suggesting specialized items that match a user's unique taste, even if those items are not popular with the general public.
- No Data Sparsity Issues: Since it doesn't rely on finding "similar users," it works effectively even when the user-item interaction matrix is mostly empty.

3. **Write disadvantages of content-based filtering approach**
   - Limited Discovery (Serendipity): The system rarely suggests items outside the user's established preferences, often creating a "filter bubble" where the user only sees things similar to what they have already consumed.
   - Feature Dependency: The quality of recommendations relies entirely on the depth and accuracy of item metadata; if the descriptions are poor or missing, the system cannot make effective matches.
   - Cold Start for New Users: Without an initial history of likes or interactions, the system has no data to build a user profile and cannot provide personalized suggestions to a new member.
   - Over-Specialization: The model tends to suggest many items that are nearly identical (e.g., suggesting ten different "Batman" movies), which can lead to a repetitive and boring user experience.
   - Inability to Capture Latent Quality: Since the system only looks at attributes (like genre or director), it cannot distinguish between a high-quality masterpiece and a low-quality project if they share the same features.

Program (Code):

Content Based Recommendation system EX11

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
df=pd.read_csv("/content/movies.csv")

#preprocessing step
df["genres"]=df["genres"].fillna("")
df["overview"]=df["overview"].fillna("")
df["content"]=df["genres"]+" "+df['overview']

#TF-IDF Vectorizer
tfidf=TfidfVectorizer(stop_words="english")
tfidf_matrix=tfidf.fit_transform(df["content"])
cosine_sim=cosine_similarity(tfidf_matrix,tfidf_matrix)
#droping the duplicates
indices=pd.Series(df.index,index=df["original_title"]).drop_duplicates()
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date: 2/4/2026**	**Enrolment No: 92301733059**

```python
def recommendation_movie(title,topn=5):
  if title not in indices:
    return "Movie is not found!"
  idx=indices[title]
  sim_scores=list(enumerate(cosine_sim[idx]))
  sim_scores=sorted(sim_scores,key=lambda x:x[1],reverse=True)
  sim_scores=sim_scores[1:topn+1]
  movie_indices=[i[0] for i in sim_scores]
  return df["original_title"].iloc[movie_indices]
```

Results:

[14] ✓ 0s

```python
recommendation_movie("Inception")
```

	original_title
2897	Cypher
4401	The Helix... Loaded
134	Mission: Impossible - Rogue Nation
1786	Flatliners
2389	Renaissance

dtype: object

[15] ✓ 0s

```python
recommendation_movie("The Lone Ranger")
```

	original_title
2357	The Legend of the Lone Ranger
1983	Meet the Deedles
1249	Torque
3450	West Side Story
3447	Butch Cassidy and the Sundance Kid

dtype: object

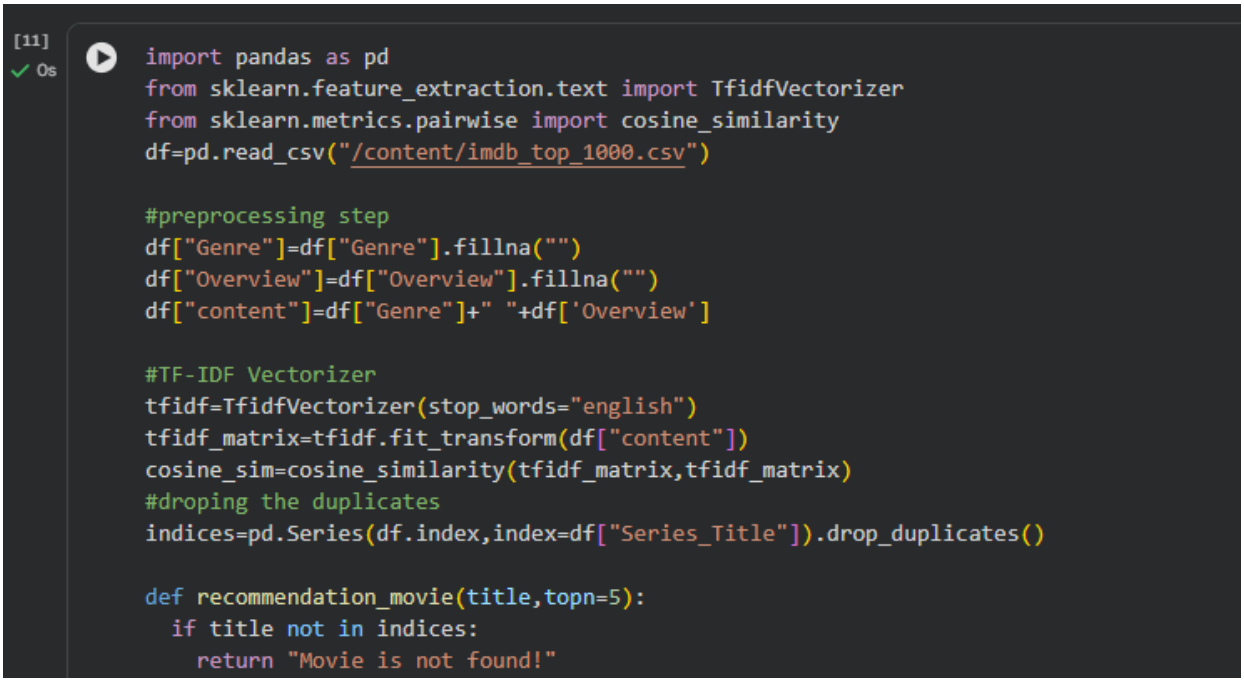
Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date: 2/4/2026**	**Enrolment No: 92301733059**

Observation:

1. Feature Dependency: Accuracy relies entirely on detailed metadata; better item descriptions lead to more relevant suggestions.
2. No Item Cold-Start: New items can be recommended immediately based on their attributes, even without any user ratings.
3. Over-Specialization: The system creates a "filter bubble," recommending very similar items and failing to provide diverse or surprising content.
4. Mathematical Relevance: TF-IDF and Cosine Similarity effectively prioritize unique keywords over common ones, ensuring a logical match.
5. User Independence: Recommendations are calculated in isolation; one user's preferences never influence another's, ensuring privacy and scalability.

Post Lab Exercise:

1. What changes you feel can be done, to improve the system that you have designed, as a part of in-house lab exercise.
2. Recommend top-10 “series” using IMDB movie dataset (Link: https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows). Attach the code with output. Also, write your observation for the same.



```
[11]
✓ 0s
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
df=pd.read_csv("/content/imdb_top_1000.csv")

#preprocessing step
df["Genre"]=df["Genre"].fillna("")
df["Overview"]=df["Overview"].fillna("")
df["content"]=df["Genre"]+" "+df['Overview']

#TF-IDF Vectorizer
tfidf=TfidfVectorizer(stop_words="english")
tfidf_matrix=tfidf.fit_transform(df["content"])
cosine_sim=cosine_similarity(tfidf_matrix,tfidf_matrix)
#droping the duplicates
indices=pd.Series(df.index,index=df["Series_Title"]).drop_duplicates()

def recommendation_movie(title,topn=5):
  if title not in indices:
    return "Movie is not found!"
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date: 2/4/2026**	**Enrolment No: 92301733059**

```
    return "Movie is not found!"
idx=indices[title]
sim_scores=list(enumerate(cosine_sim[idx]))
sim_scores=sorted(sim_scores,key=lambda x:x[1],reverse=True)
sim_scores=sim_scores[1:topn+1]
movie_indices=[i[0] for i in sim_scores]
return df["Series_Title"].iloc[movie_indices]
```

```
[12] recommendation_movie("Inception")
```

	Series_Title
654	Gattaca
353	Nefes: Vatan Sagolsun
708	À bout de souffle
645	Wo hu cang long
412	Kagemusha

dtype: object

```
[17] recommendation_movie("avatar")
'Movie is not found!'
```

```
[18] recommendation_movie("Pulp Fiction")
```

	original_title
3526	The Sting
3194	All or Nothing
4624	Locker 13
3466	Sliding Doors
3504	11:14

dtype: object

Critical Analysis

While the system is excellent at finding "more of the same," it will likely observe **Over-Specialization**. For example, if a user watches one Anime, the system might fill the entire homepage with Anime, failing to realize the user might also like a high-quality Documentary. This is known as the **Filter Bubble**.